



Home > ポートフォリオ

Tweet

Follow



ポートフォリオ

最終更新日

2023 年 8 月 16 日



プロフィール



氏名

石岡隆光

メール

ishioka@starship.jp

電話

コンタクトページからお問い合わせください

住所

埼玉県川口市

詳しくはコンタクトページからお問い合わせください

最終学歴

国立東京外国語大学 英米語学科卒業（国際関係論、言語学、翻訳、法学、経済学）

職歴

以下に主なものを挙げます



概要

通産省の機械翻訳プロジェクト（通称 Mu プロジェクト。場所は京大の長尾研）

期間

1983～1985

ハードウェア

富士通のメインフレーム（京大計算機センター）
Symbolics 社の Lisp マシン

OS

不明（当時の OS はハードウェアと一体になっていました）

言語等

UTI-LISP（東京大学が開発した LISP の方言）

業務内容

1. 英日・日英翻訳アルゴリズムの開発

役割

1. 英語解析部の設計、実装、テスト
2. 日英トランスファー部の設計、実装、テスト





概要

NEC の機械翻訳プロジェクト（最初の 2 年は中央研究所。その後は三田の事業部）

期間

1985～2000

ハードウェア

VAX

NEC 製 UNIX ワークステーション

NEC PC-9801 シリーズ（当時の普通の MS-DOS パソコンです）

OS

BSD UNIX (VAX)

System V UNIX（ワークステーション）

Windows NT, Windows 95（パソコン）

言語等

C (cc), Visual C/C++, Visual Basic, Franz Lisp, UTI-LISP, csh, awk, perl

業務内容

1. 英日・日英翻訳アルゴリズムの開発
2. 英日・日英翻訳辞書の開発
3. 開発ツールの開発

ソースコードバージョン管理、一括大量翻訳テスト自動化、品質評価半自動化、課題追跡、コミュニケーションなどのツール
今なら DevOps や CI/CD と呼ばれる体制を支えるツール群を自前で作成・運用していました

役割

1. 英語解析部の基本設計から実装、テストまで
2. 辞書開発チームのリーダー
3. 開発ツールの企画、設計、実装、テスト、運用





概要

日本語構文解析システムの開発（初め東京ソフト株式会社。後にシステム部がスピンオフ）

期間

2000～2009

ハードウェア

IBM PC-AT 互換機（今の普通のパソコンです）

OS

FreeBSD, Linux (Fedora), Windows, Windows Server

ミドルウェア

httpd (Apache)

言語等

Visual C/C++, C++ (gcc), C#, Java, HTML

業務内容

1. 日本語構文解析ライブラリの開発

大きな木構造の中から小さな木構造を探し出し、見つけた木構造を任意の別の木構造に変換する汎用 Tree Transducer ライブラリを開発 (Visual C++)。

この種の Tree Transducer はヘタに作ると非常に遅くなるのですが、私はそれを防ぐために

1. フォーカス
2. トラバース

という概念を導入し、高速化に成功しました。

これは京大で使っていた GRADE という Tree Transducer と NEC のシステムの長所を合わせたものです。

それを使ってルールベースの構文解析システムを開発しました。

2. 構文解析結果をグラフィカルな木構造で表示するウェブデモシステムの開発 (Java Applet)

訪問者が任意の日本語を入力すると、それを形態素・構文解析して結果をグラフィカルな木構造として表示するウェブサイトを開発。

それを見た野村総研さんから引き合いがあり、同社の CRM 製品 TRUE TELLER に採用されました（今は別のものに置き換えられています）。

その後、それがシステム部門の収益の柱になり、後にシステム部門が独立して別の会社になりました。





3. 日本語形態素解析ライブラリの開発 (Visual C++)

形態素解析の多数の解釈を、解釈の分岐点で複数の形態素ノードに分岐する有向グラフとして表現し、その有向グラフが含む多数の経路のコストを評価、比較して最もコストが低いものを選ぶライブラリを開発しました。

また、その有向グラフをグラフィカルに表示し、ユーザーが分岐点で「正解」のアーク（エッジ）をクリックすることでコストを調整する開発ツールを開発しました。

4. それらのシステムを Linux に移植

Apache + CGI + REST API でウェブサービス化



5. OCR 結果修正支援ツールの開発

東京ソフトさんの本業であるレセプト入力を支援するツール (Visual C++)

元データである手書き文書の画像データと、それを OCR で認識した座標付きテキストデータを画面の左右に並べて表示します。

テキストデータの領域はエディターになっていて、そこでカーソルを動かすと、カーソル位置の文字に対応する画像の位置をハイライトします。

つまり、「この文字は画像のどこを認識した結果なのか」を分かりやすく表示します。

これは大いに生産性向上に寄与したらしく、「システム部が作ったツールで役に立つものを初めて見た」と入力業務の責任者からほめられました。

役割

1. 製品の企画、設計、実装、テスト、運用
2. ウェブデモサイトの設計、実装、テスト、運用
3. ネットワークの設計と構築
4. Linux のインストールと設定（当時は仮想マシンではなく実機でした）
5. エンジニアチームの教育、指揮、監督



概要

株式会社エクスリンクのショートメッセージ一括大量送信システムの開発

期間

2010～2022

ハードウェア

AWS

OS

Linux (Amazon Linux, Amazon Linux 2)

ミドルウェア

Apache, RDS (PostgreSQL), Aurora (PostgreSQL)

言語等

C++ (gcc), HTML, css, javascript, jquery, bootstrap, PHP, bash, Java

業務内容

(2010～2014 は同社で動画配信サービスや音声合成 Web API サーバーや動画・音声を合成するキャンペーンサイト構築等の作業をいくつかしましたが、細かい受託開発なので割愛します)

エクスリンクさんは元々ショートメッセージ一括送信システムを持っていたが、そのシステムには次のような問題がありました。

1. jsmpp (Java の SMPP ライブラリ) に頼り切った高速化不能の送信モジュール
Java 製 ORM マッパー Hibernate がすべての DB テーブルをロックしてしまうためマルチプロセス化不能、そのため、
 1. それがボトルネックとなってスループットが低いところで頭打ちになり
 2. また、単一障害点となってしばしばシステム障害を引き起こしていた
2. サーバーサイド (php) で HTML を動的に生成するタイプの、アーキテクチャも見た目も古い管理画面 GUI
3. 公開 API が php で実装されていたため、リクエストのたびに DB を検索せざるを得ず、スループットが低い
4. 継続的な開発と運用の仕組みもツールも存在せず、すべてをプログラマーの手作業と注意力のみに依存していたため、しばしば障害が発生





5. Java の宿命である GC によるスループットの異常な低下が定期的に発生。そのたびにプログラマーが手動で送信系プロセスを再起動
6. DB の送信ログのレコード数が増えるとスループットが低下。そのたびにプログラマーが手動で DB テーブルのレコードを削除

私はそれらの問題を解決するために、全てを概念設計からやり直し、全く新しく実装しました。

1. 送信モジュールを高速化

1. C++ (socket + pthread) で生の SMPP 通信プロトコルを実装
1 SMPP アカウント 1 プロセスでマルチプロセス化し、
submit_sm, deliver_sm の送受信処理を並列化
2. 処理済みの DB レコードは別のテーブルに移動

それらにより、スループットは旧システムの 10 倍になりました。

また、定期的なスループットの低下もなくなりました。

このアーキテクチャならば、SMPP の契約数とプロセス数（必要なら仮想マシン数）を増やせば、ほぼそれに比例してスループットが大きくなるので、今後もいくらでも拡張可能です（ボトルネックは DB のみ。これについてはデータベース・サーバーのスペックを上げずにボトルネックを解消する案があります）。

2. 管理画面を ajax モデルで一新

1. サーバースクリプト (php) は HTML の動的生成を行わず、スタンディックな HTML と javascript を返すだけ
ただし、#include による画面要素の部品化は行う
2. サーバーは API コマンドを処理する。API コマンドの中身はほぼ DBMS との通信と結果の加工と JSON 化
生の php (フレームワーク無し) で実装
3. クライアントは生の javascript と、jquery, bootstrap と某モダンなテンプレートで実装
4. draggable ダイアログ、D&D で列の順序を変えられる表、問題の箇所に表示されるトゲ付き noty 風ツールチップ、モダンなドロップダウンリスト等、既成の部品の挙動がイマイチだったものは自作

3. 公開 API を C++ で実装。DB のデータをメモリにキャッシュすることでスループットを数十倍に

4. 管理画面と送信モジュールのシステムをそれぞれ「開発用」「商用β」「商用」の 3 系統作成

5. それらの間の差分表示、dry run モードでの更新ファイル確認、実際の更新を行う bash スクリプトを作成

今はこの体制のことを CI/CD (Continuous Integration and Continuous Delivery) と呼ぶようですが、当時は（少なくとも日本では）あまり意識されない概念だったような気がします





6. 以上を実現するために DB のテーブル定義を全て新たに設計
旧システムから新システムの DB に一括変換するスクリプトを作成
つまり、旧システムから流用したのは顧客のデータだけです

役割

以上の作業を、DB と送信モジュールと公開 API については設計から実装、テストまで私一人で実行

GUI と内部 API も大枠は私が作り、generic.html および php の function Generic(\$args) を作成して他のプログラマーに使ってもらうようにしました

枠の中身をデバッグし充実させることと、新機能を追加することは他の人たちに手伝ってもらいました





概要

個人で Linux 用 IDS (Intrusion Detection System) TCPDetective を開発

期間

2022～現在

ハードウェア

AWS

OS

Amazon Linux 2

ミドルウェア

Apache, RDS (PostgreSQL)

言語等

SystemTap, C++ (gcc), PHP, bash

業務内容

1. Linux のカーネルモードで IP パケットをキャプチャし、同時にそのパケットに関連するプロセスとそのコマンドラインを特定・記録するプログラム TCPDetective を開発
詳細はこちら→<https://www.tcpdetective.com/>
(<https://www.tcpdetective.com/>)

役割

企画、設計、実装、テスト、運用まですべて





概要

前記の IDS (Intrusion Detection System) TCPDetective を PayPal の「サブスクリプション」で販売する EC サイトを開発

期間

2022～現在

ハードウェア

AWS

OS

Amazon Linux 2

ミドルウェア

Apache, RDS (PostgreSQL)

言語等

C++ (gcc), HTML, css, javascript, jquery, bootstrap, PHP, bash, node.js

業務内容

このウェブサイトの構造と機能

1. クライアント（管理画面）

1. システム管理者用

1. ログイン、ログアウト、プリファレンス（例：言語）管理
2. ダッシュボード（PayPal API 操作）
 1. Product 管理（一覧、作成、編集）
 2. Plan 管理（一覧、作成、編集）

3. サポートセンター機能

2. 一般ユーザー用

1. ユーザー登録、ログイン、ログアウト、プリファレンス（例：言語）管理
2. ダッシュボード（サブスクリプション管理）
 1. サブスクリプション一覧
 2. サブスクリプション操作 (PayPal API を内部 API サーバー経由で呼び出す)
3. ユーザーガイド
4. サポートセンター機能

3. ゲストユーザー用

1. ユーザー登録、ログイン、ログアウト、プリファレンス（例：言語）管理





2. ユーザーガイド
2. サーバー（内部 API）
 1. クライアント（管理画面）が提供する機能のうち、「プリファレンス管理」以外のすべてが内部 API 呼び出しによって実現されている
 2. 管理画面サイトの特定のパスへのアクセスを Apache の管理画面 VirtualHost 内に定義された ProxyPassReverse によって内部 API サイトへ redirect
 3. 内部 API の実体は、生の php で記述した REST API レイヤー
 1. x-www-form-urlencoded でリクエストを受け取り、多くの場合 DB クエリを発行し、結果を JSON 形式で返す
 2. PayPal API の呼び出しも、すべて内部 API 経由で行う（javascript はデータを隠蔽できないので PayPal API の情報を書くのは危険）
 3. DB, Curl, Mail, LINE などの主要な部品のクラスは自作
3. サーバー（公開 API）
 1. ユーザーのホストにインストールされた TCPDetective デーモンの状態を管理する
 2. 公開 API の実体は、生の php で記述した REST API レイヤー
 1. TCPDetective から x-www-form-urlencoded でリクエストを受け取り、DB にログを書き込み、結果を JSON 形式で返す
 2. 上記の一般ユーザー用管理画面のところで説明したユーザーのホスト上の TCPDetective デーモンの操作（自動停止、起動不能、起動可能）は、この公開 API で実現している

このウェブサイトの特徴

1. クッキーをまったく使わない。従って当然、「疑似セッション」の仕組みも使っていないので、LB での特別な配慮が不要、また、セッションのタイムアウトも無い
2. ユーザー認証は API 呼び出しのたびに毎回行う
3. まったくページ遷移をしない、いわゆる SPA (Single Page Application) のフレームワーク（自作）を使用
 1. そのおかげで「状態遷移」をクライアントのメモリ上で行うことができる
つまり、ウェブアプリでありながら、昔からある PC やスマホのアプリケーションと同じ造りになった
そのため、見かけのページがどんなに変わっても、ユーザー情報や加工中の中間的データをそのページで使うことができる
これによりウェブアプリの特殊性が消え、非常にプログラムが書きやすくなった
 2. サーバー側で HTML の動的生成をまったく行わないので、サーバー側のフレームワークもスクリプトも必要としない。当然、「デ



ンプレート」も無い

サーバー側にはただ静的な html ファイルと javascript があるだけ

3. サイトを構成するページ群の HTML の内部構造に関して、クライアントはまったく何の想定もしない。どんな構造の HTML でも表示可能

そのため、デザイナーさんが作成した「サーバーサイドスクリプトを想定しないただの HTML」をそのまま使用することができる

4. このフレームワークは、/index.html と /js/common.js と /js/website.js によって実現されている。ライブラリは jquery 要求されたページの html を \$.ajax() で取得し、取得した html の中に

```
<!-- __INCLUDE__ /foobar.html -->
```

という「スロット」があれば、その html を再帰的に取得し、取得した html の文字列でそのスロットを置換する

特殊な記法も、jquery 以外のライブラリの知識も不要

4. クライアント（管理画面）、内部 API、データベース、公開 API、TCPDetective ビルド環境、tarball ビルドデーモンのセットが、「開発」「商用β」「商用」の 3 セット作成してある（ただしデータベースは「商用β」と「商用」が同じもの）

それらの間の差分表示、--dry-run モードによる更新ファイルのリスト表示、実際の更新を行う bash スクリプト

役割

企画、設計、実装、テスト、運用まですべて

上記の各プロジェクトに従事していた間に、所属企業も契約形態も何回も変わっています（派遣、正社員、自営）が、それらを書くともしろ分かりにくくなるので割愛しました。「派遣」も「正社員」も業務を委託する側の方便であり、実態はすべて「何かを石岡に作らせる」でした。



「システムとは何か」に対する深い洞察と、それを基盤とした演繹的推論の能力があります。



「手持ちの道具を組み合わせることができることをやる」

のではなく

「まず目的を定め、それを実現するために何が必要かを考える」

そして

『無いものは作る』



また、能書きを垂れるだけではなく、実装の過程で未知の技術が必要になれば、それをその場で習得しつつ、実際に動き、役に立つものを作ってきました。

必要な技術はすべて必要性を感じてから習得してきたので、GCP、TypeScript、Python、機械学習等、今後必要になる未経験の技術も短期間で使いこなす自信があります。

例: 今、GCP で TensorFlow2 を使っています（既に理解したつもりになっています）。

例: 最近、Codecademy で TypeScript のコースを始めました（既に理解したつもりになっています）。

例: ChatGPT-4 は既に手放せない日常のツールになっています。この経歴書も ChatGPT と相談しながら書きました。

必要に応じて中～小規模のライブラリやフレームワークなら自分で作ることができます。

また、長い間に作り溜め、動作検証済みの自作ライブラリを持っています。

例: C++ のメモリ断片化を防ぐメモリ管理ライブラリ。これは、日本語構文解析ライブラリ、ショートメッセージ括送信システムの送信モジュールと外部 API、および TCPDetective に使いました。

例: <https://www.tcpdetective.com/> は自作の SPA (Single Page Application) フレームワークで作りました。

ここまでの文章を読まれると「理屈屋で気難しい性格」を想像されるかもしれませんが、実は硬軟のコミュニケーションにも長けていると思います。

https://twitter.com/kisaburo_y では人格をほぼ丸出しにしています。

Windows用メールソフト『電信八号』で「第5回フリーソフトウェア大賞」に入賞しました。

<https://www.iajapan.org/osp/menu04.html>